

Návrh zápočtového programu

Jan Matula

15. září 2026

- **Student:** Jan Matula
- **Předmět:** Programování 2
- **Cvičící:** Jiří Šejnoha

Základní popis projektu. Cílem projektu je knihovna pro základní práci s klasickou výrokovou logikou. Mezi základní abstrakce, které by knihovna měla poskytovat, patří: (1) Definice příslušných syntaktických a sémantických objektů, zejména výroků a valuací. (2) Funkce pro základní syntaktické manipulace s těmito objekty: “foldování” výroků, “mapování” atomů (ve smyslu funkcionálního programování), dosazování za atomy atd. Dále konverze výroků na ekvivalentní výroky příslušící fragmentům: konverze do “negation normal form”. (3) Funkce pro základní sémantické operace: vyhodnocování výroků podle valuace, testování ekvivalence, testování splnitelnosti a ekvisplnitelnosti (tyto operace ovšem budou používat neefektivní “brute force” algoritmus) atd. (4) Definice formátu pro čitelný zápis výroků, funkce, které převede výrok na zápis, a opačná funkce, která převede zápis na výrok.

Dále by knihovna měla poskytovat funkci pro efektivní testování splnitelnosti výroků: úloha SAT. Přesněji: (5) Definice výroků v “conjunctive normal form” (CNF) společně s funkcí pro konverzi libovolných výroků na ekvisplnitelné výroky v CNF. (6) Funkce poskytující efektivní řešení úlohy SAT pro výroky v CNF. Zbytek dokumentu se zabývá touto poslední částí projektu, jelikož je z pohledu algoritmizace nejnáročnější.

Reprezentace úlohy SAT. Literál je výrok tvaru v nebo $\neg v$, kde v je výroková proměnná. Literál prvního tvaru nazýváme pozitivním literálem, druhého negativním literálem. Na literálech je užitečné definovat $(v)^\neg = \neg v$ a $(\neg v)^\neg = v$. Klausule je disjunkce literálů, tedy výrok tvaru $(l_1 \vee l_2 \vee \dots \vee l_n)$, kde $l_1, \dots, l_n$ jsou literály. Výrok v CNF je konjunkce klauzulí. Nechť $\mathcal{L}$ je množina všech literálů. Podmnožina $V \subseteq \mathcal{L}$ je valuace, pokud pro každou proměnnou v obsahuje nejvýše jeden z literálů v a $\neg v$. Valuce V splňuje klauzuli $(l_1 \vee l_2 \vee \dots \vee l_n)$, pokud obsahuje alespoň jeden z literálů l_i , a splňuje výrok v CNF, pokud splňuje všechny jeho klauzule. Výrok v CNF je splnitelný, když existuje nějaká valuace, která ho splňuje. (Protože výrok obsahuje pouze konečně mnoho proměnných, stačí uvažovat pouze konečné valuace.)

Vstupní výrok budeme reprezentovat jako objekt typu `list[list[int]]`. Předpokládejme, že proměnné jsou očíslované $v_1, v_2, v_3, \dots$. Potom kladný literál v_i reprezentujeme jako kladné číslo i , záporný literál $\neg v_i$ jako záporné číslo $-i$. Prvky typu `list[int]` reprezentují klauzule. Valuace reprezentujeme

jako objekty typu `dict[int, bool]`. Výstupem bude tedy objekt typu `tuple[bool, dict[int, bool] | None]`. Výstup tvaru `(False, None)` znamená, že vstupní výrok je *nesplnitelný*, zatímco `(True, vln)` znamená, že je *splnitelný* a `vln : dict[int, bool]` je splňující valuace.

DPLL algoritmus. Necht K je množina klauzulí. Valuace V splňuje K , pokud splňuje všechny příslušné klauzule; množina K je splnitelná pod V pokud existuje valuace $V' \supseteq V$ taková, že V' splňuje K . Základem algoritmu jsou následující pravidla: (1) Pokud $(l_1 \vee l_2 \vee \dots \vee l_n) \in K$ a $l_1^\neg, \dots, l_n^\neg \in V$, potom K není splnitelná pod V . (2) Pokud $(l_1 \vee l_2 \vee \dots \vee l_n) \in K$ s tím, že pro nějaké $k \in [1, n]$ máme $l_k^\neg \notin V$ a $l_1^\neg, \dots, l_{k-1}^\neg, l_{k+1}^\neg, \dots, l_n^\neg \in V$, potom K je splnitelná pod V právě, když je splnitelná pod $V \cup \{l_k\}$. (3) K je splnitelná pod V právě, když je splnitelná pod $V \cup \{l\}$ nebo pod $V \cup \{l^\neg\}$.

Tato pravidla zhruba definují algoritmus DPLL. Pomocí pravidel (1), (2) se snažíme rozšiřovat valuaci V dokud nenalezneme konflikt. Jakmile V nelze rozšířit pravidlem (2), musíme použít pravidlo (3): Pokusíme se K splnit pod $V \cup \{l\}$; pokud to nelze (pokus vede ke konfliktu), “vrátíme se” (“backtracking”) a pokusíme se ji splnit pod $V \cup \{l^\neg\}$; pokud to také nelze (také vede ke konfliktu), vidíme, že K není splnitelná pod V .

To, zda lze aplikovat pravidlo (1) nebo (2), lze detekovat, aniž bychom v každé iteraci kontrolovali každou klauzuli. Z každé klauzule vybereme dva literály l tak, aby pro oba platilo $l^\neg \notin V$. (Detail: sledované literály mohou být vybrány tak, že pro jeden $l_1 \in V$ a pro druhý $l_2^\neg \in V$ s tím, že l_1 byl přiřazen na stejné “úrovni větvení” jako l_2 .) Každá klauzule v K má takto dva “sledované literály”, které zaručují, že na ni nelze aplikovat ani jedno z pravidel. Když V rozšiřujeme na $V \cup \{l\}$, stačí zkontrolovat klauzule, ve kterých je $l^\neg$ sledovaným literálem, a případně v těchto klauzulích najít náhradní sledované literály.

CDCL algoritmus. DPLL algoritmus můžeme vylepšit tím, že implementujeme “conflict-driven clause learning” (CDCL). Konflikt vzniká, když máme literál l a dvě klauzule $(l_1 \vee \dots \vee l_n \vee l)$, $(l'_1 \vee \dots \vee l'_m \vee l^\neg) \in K$ takové, že $l_1^\neg, \dots, l_n^\neg, l'_1, \dots, l'_m \in V$. Z konfliktu můžeme odvodit lemma $(l_1 \vee \dots \vee l_n \vee l'_1 \vee \dots \vee l'_m)$ a přidat ho mezi klauzule. Pokud k přiřazení $l_1^\neg \in V$ došlo podle pravidla (2), máme další klauzuli $(l_{11} \vee \dots \vee l_{1p} \vee l_1) \in K$ takovou, že $l_{11}^\neg, \dots, l_{1p}^\neg \in V$. Protože tato přiřazení vnucují $l_1^\neg \in V$, můžeme alternativně odvodit lemma $(l_{11} \vee \dots \vee l_{1p} \vee \dots \vee l_n \vee l'_1 \vee \dots \vee l'_m)$ atd. Tímto postupem můžeme odvodit klauzuli “přiměřené” velikosti a “vrátit se” na “vhodnou” úroveň.

Testování implementace algoritmu. Výsledky algoritmu můžeme na dostatečně malých, náhodně generovaných výrocích porovnávat s výsledky získanými “brute force” metodou.